UNIVERSITY OF LEEDS

COMP1850 Programming

Week 2, Session 1

Python's Collection Types

UNIVERSITY OF LEEDS

Recap

Last week we covered

- Python's numeric and string data types
- Variables and expressions
- Printing things in Python programs
- Reading input from the terminal

UNIVERSITY OF LEEDS

Today's Aims

- Consider why it is useful to group items of data together and manipulate them as a single collection
- Recognize that we can collect items together in different ways
- Understand how to use Python's built-in collection types

UNIVERSITY OF LEEDS

Example: ‘To Do’ Lists

When we have a sequence of tasks that we want to remember:

- We collect them all together in a single list
- We arrange them in a specific order: the order in which we want to carry out the tasks

How can we represent this in a Python program?

UNIVERSITY OF LEEDS

Example: Favourite Movies

Suppose I ask you all what your favourite movie is, and I collect together your answers...

- I'm not interested in ordering the answers in any particular way
- I don't want to see "Barbie" appearing multiple times – i.e., I want to ignore duplicates in the collection

UNIVERSITY OF LEEDS

Example: Elections

The Student-Staff Committee wants to elect a student member to represent Computer Science Level 1, and there are multiple candidates. We need to tally all the votes to see who wins.

- Order of candidate names doesn't matter
- Collection needs to hold two different pieces of data for each candidate:
  - Their name (string)
  - Number of votes cast for them (integer)

UNIVERSITY OF LEEDS

Python Collections

Python has many collection types, including

- Lists
- Tuples
- Sets
- Dictionaries

These all have different properties, and therefore different uses...

Lists

A list is an ordered, one-dimensional array of objects

We can create a list using square brackets, with a comma to separate each item of data:

```
numbers = [4, 2, 7, -1]

things = [-1.2, "duck", 4, "Q"]

data = []
```

mixed data types

List items are usually of the same type, but they don't have to be

UNIVERSITY OF LEEDS

List Indexing

The position of an item in a list is called the **index**

Consider this list:

```
a = [4, 2, 7, -1]
```

- The first item is at index 0 and we count from there
- We can extract individual items from the list using the index:

```
print(a[0])    # prints 4
print(a[1])    # prints 2
```

Modifying Lists

Lists are dynamic and mutable data structures:

- An item can be added to the end or at a specified index
- An item can be removed from the end or at a specified index
- An item at a specified index can be replaced

These operations are implemented as **methods** of the `list` type...

Some List Methods

UNIVERSITY OF LEEDS

Method	Purpose
`append(item)`	Adds an item at the end of the list
`insert(index, item)`	Adds an item at the specified position
`pop(index)`	Removes the item at the specified position (or last item if no index provided)
`remove(item)`	Removes the first item that has the specified value
`count(item)`	Returns number of occurrences of specified item in the list
`index(item)`	Returns the index of the first item that has the specified value
`clear()`	Removes all the items from the list
`reverse()`	Reverses the order of the list
`sort()`	Sorts the items in the list into their natural order

UNIVERSITY OF LEEDS

List Uses

Lists are a very common feature of Python programs:

- Their dynamic nature makes them very flexible
- Manipulation is easy, using list methods
- It is easy to iterate over contents of a list (see next week)
- They can be used to represent different kinds of data structure – e.g., **stacks** and **queues**

UNIVERSITY OF LEEDS

Task 1: Shopping List

`week_02/session_1/tasks/`

1. Edit `task_1.py`
2. Run the program and inspect its output
3. We want to buy grapes instead of bananas: replace the relevant item in the list, using its index
4. We want to buy yoghurt, which is near milk in the shop: insert `"yoghurt"` into the list, next to `"milk"`

UNIVERSITY OF LEEDS

Task 2: Item Ordering & Removal

`week_02/session_1/tasks/`

Add lines of code to `task_2.py` that

- Sort the list alphabetically
- Reverse the order of items in the list
- Remove all items from the list

Add a print statement after each line, to see what happens

UNIVERSITY OF LEEDS

Comparison With Strings

Strings are not lists, but they are similar in a couple of ways:

- A string is an ordered sequence of items, like a list
- We can use indexing to access those items:

```
text = "a string"
print(text[4])      # what does this print?
```

Strings are immutable: we cannot alter the contents of a string (but we can generate modified versions using string methods)

UNIVERSITY OF LEEDS

Tuples

A **tuple** is an ordered, one-dimensional array of objects

We can create a tuple using parentheses (round brackets), with a comma to separate each item of data:

```
album = ("Taylor Swift", "Folklore", 2020)
```

Tuples are immutable data structures: after creation, we cannot add, remove or replace items

UNIVERSITY OF LEEDS

Operations on Tuples

- We can do indexing (to read item values, not replace them)
- We can use `count()` and `index()` methods, as we would for lists
- We can 'unpack' tuple items into separate variables:

```
t = (1, 2, 3)   # packs data into tuple
(a, b, c) = t   # unpacks tuple into variables
print(b)        # what is printed here?
```

UNIVERSITY OF LEEDS

Task 3: Manipulating Tuples

`week_02/session_1/tasks/`

Add code to `task_3.py` that

- Finds the position of `"banana"` in the tuple
- Counts the number of occurrences of `"cherry"`
- Unpacks the tuple into variables `first`, `second`, `third`

Add suitable print statements to the program, to help you check whether these operations work correctly

UNIVERSITY OF LEEDS

Sets

A **set** is unordered collection of unique items

- Items cannot be indexed, but we can test whether an item is present
- Items can be added, but duplicates will be ignored
- Items can be removed

```
fruit = {"apple", "orange", "banana"}    # note use of {}

data = set()    # empty are sets created differently!
```

UNIVERSITY OF LEEDS

Set Uses

Sets mimic mathematical sets, so they are useful in any situation where the solution to a problem can be expressed with set theory

A common application is the removal of duplicates from a list:

```
data = ["apple", "orange", "apple", "grape"]

unique_data = list(set(data))
```

Try this out!
What do you notice about the results?

Some Set Methods

UNIVERSITY OF LEEDS

Method	Purpose
`add(item)`	Adds specified item to the set
`discard(item)`	Removes specified item from the set
`union(setB)`	Returns new set containing elements of the set and those of `setB`
`intersection(setB)`	Returns new set containing elements of the set that are also in `setB`
`difference(setB)`	Returns new set containing elements of the set that are not in `setB`
`symmetric_difference(setB)`	Returns the complement of `intersection(setB)`

UNIVERSITY OF LEEDS

Task 4: Set Operations

`week_02/session_1/tasks/`

1. Examine the code in `task_4.py` and run the program
2. Add an item to the set named `fruit`
3. Remove an item from the set named `vegetables`
4. Compute and display the symmetric difference of the two sets

Dictionaries

UNIVERSITY OF LEEDS

A **dictionary** is a collection of *key*: *value* pairs

- We look up values using the key, rather than an index
- Value associated with a key can be replaced
- Pairings of key & value can be added or removed

```
prices = {"apple": 21, "orange": 30, "banana": 45}

data = {}
```

UNIVERSITY OF LEEDS

Dictionary Operations

```
prices = {"apple": 21, "orange": 30, "banana": 45}

print(prices["apple"])   # prints 21
prices["apple"] = 25     # replaces value for key 'apple'
print(prices["apple"])   # prints 25
print(prices["kiwi"])    # what happens here?
prices["kiwi"] = 53      # adds new key-value pair
```

Some Dictionary Methods

UNIVERSITY OF LEEDS

Method	Purpose
`clear()`	Removes all key-value pairs from the dictionary
`get(key)`	Returns the value associated with the specified key
`items()`	Returns a sequence of tuples containing keys & associated values
`keys()`	Returns a sequence containing the dictionary’s keys
`pop(key)`	Removes the key-value pair for the specified key
`values()`	Returns a sequence of the dictionary's values

Dictionary Uses

UNIVERSITY OF LEEDS

Dictionaries can be useful when

- Data items don't form a sequence, so we can't use position (index) to access individual values
- We want to look up values using something other than an integer

Examples:

- Address book (strings as keys & values)
- Tally of votes in an election (string keys, integer values)

UNIVERSITY OF LEEDS

Task 5: Dictionaries

`week_02/session_1/tasks/`

1. Examine the code in `task_5.py` and run the program
2. Add a new entry, `"York":"Ouse"`, plus one other of your choice
3. Display all the keys
4. Display all the values
5. Display the key:value pairs as tuples

Combining Collections

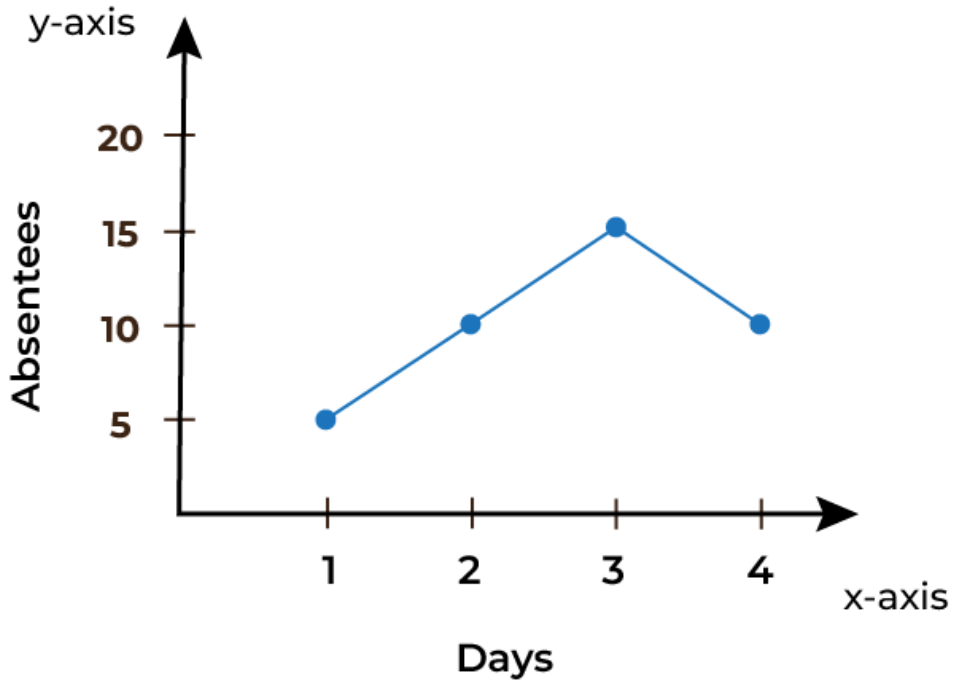
UNIVERSITY OF LEEDS

We can combine these basic collection types to create more complex data structures:

- Lists can contain tuples, or other lists
- The value associated with a dictionary key can be a list
- Tuples can be used as the keys of a dictionary
- etc...

With careful design this allows us to model a very wide range of real-world data

Example



How could you combine Python's collection types to represent this line graph?

Write down the actual data structure that represents the line

UNIVERSITY OF LEEDS

Task 6: Nested Collections

`week_02/session_1/tasks/`

- In `task_6.py`, create a database of your favourite music
- Use a dictionary to represent the entire database, with artist names as the keys
- The values associated with a key should be a list, containing the titles of albums released by that artist
- Use the `pprint()` function to ‘pretty print’ the data structure

Collection Properties

UNIVERSITY OF LEEDS

	List	Tuple	Set	Dict
Notation	[]	()	{}	{}
Ordered	Yes	Yes	No	Yes (since Python 3.7)
Duplicates	Yes	Yes	No	Key: No Value: Yes
Mutable	Yes	No	The set: Yes Values: No	Key: No Value: Yes

UNIVERSITY OF LEEDS

Useful Functions

- Python has a number of useful built-in functions that can be used with collections of different types
- `len()` will tell you the size of a collection:
- Try invoking `min()` and `max()` on a tuple or list of numbers, then try the same with a tuple or list of strings
- Do `min()` and `max()` work with sets and dictionaries? Try it!

Summary

We have

- Explored why we collect multiple data items into a single object
- Introduced four Python collections types: list, tuple, set, dict
- Seen examples of how to use these collection types
- Noted that these types can be combined to create arbitrarily complex data structures